

Day 109: Decision Trees and Random Forest Basics

1. Day number

Day 109

2. Topic name

Decision Tree and Random Forest

3. Connection

Previous classifiers used different ideas:

- **Logistic Regression** → learned a classification boundary.
- **KNN** → looked at nearby examples.
- **Naive Bayes** → used probability.

Today, you will learn a very intuitive approach:

| **Make a prediction by following a sequence of simple decisions.**

This is the basic idea behind a **Decision Tree**.

4. Important topics

Root

The **root** is the first decision at the top of the tree.

Example:

Is monthly_usage > 50?

Every example starts here.

Split

A **split** is a decision that separates data into groups.

For example:

monthly_usage > 50?

creates two groups:

Yes
No

Branch

A **branch** is one possible path after a decision.

```
usage > 50?  
  /      \  
Yes      No
```

The two lines are branches.

Leaf

A **leaf** is the final prediction at the end of a path.

For example:

Leaf → No Churn

or:

Leaf → Churn

Decision Tree

A **Decision Tree** combines several simple decisions.

Conceptually:

```
Question  
  ↓  
Question  
  ↓  
Question  
  ↓  
Prediction
```

Multiple trees

Instead of relying on just one tree, we can train several trees.

Each tree may make its own prediction.

Random Forest

A **Random Forest** combines the predictions of many Decision Trees.

For classification, the trees can roughly vote:

```
Tree 1 → Class A  
Tree 2 → Class B  
Tree 3 → Class A  
Tree 4 → Class A  
Tree 5 → Class B
```

Majority:

```
Class A = 3
Class B = 2
```

Final prediction:

```
Class A
```

5. Foundational notes

Decision Trees are useful because their reasoning can often be understood as a sequence of questions.

Suppose we want to predict whether a customer will churn.

Features:

```
monthly_usage
support_calls
```

Target:

```
churn
```

where:

```
0 = No Churn
1 = Churn
```

A simplified tree might behave like this:

```
Is monthly_usage < 40?
  |
  +---+---+
  Yes   No
  |     |
support No Churn
calls > 3?
  /   \
Yes    No
|      |
Churn  No Churn
```

The real algorithm chooses useful splits from the training data, but you do **not** need the splitting mathematics yet.

At beginner level, think:

| A Decision Tree learns a useful sequence of yes/no questions.

6. Decision Tree using your early Python if-else thinking

You already learned something very similar when you studied Python conditionals.

Imagine this Python-style logic:

```
if monthly_usage < 40:
    if support_calls > 3:
        prediction = "Churn"
    else:
        prediction = "No Churn"
else:
    prediction = "No Churn"
```

That is very close to how a simple Decision Tree can be understood.

The difference is important:

With normal Python:

```
You manually write the conditions.
```

With machine learning:

```
The Decision Tree learns useful conditions from training data.
```

So:

```
Python if-else
→ programmer chooses rules

Decision Tree
→ algorithm learns rules from examples
```

That connection makes Decision Trees especially intuitive.

7. Visual idea

Suppose a tree has learned questions involving two features.

A new point starts at the **root**, answers each question, follows the corresponding **branch**, and eventually reaches a **leaf** containing its predicted class.

The important thing to notice is that the model does not check every possible path. One example follows **one path from root to leaf**.

8. Random Forest — many trees working together

A single Decision Tree can sometimes rely too heavily on particular details in the training data.

Random Forest improves the basic idea by creating **many Decision Trees**.

Imagine asking one person:

Will this customer churn?

One person's answer could be wrong.

Instead, imagine asking 100 people who each looked at somewhat different information.

You collect their votes:

60 → Churn
40 → No Churn

The group predicts:

Churn

A Random Forest works somewhat like that.

Conceptually:

```
Training data
  ↓
Tree 1 → prediction
Tree 2 → prediction
Tree 3 → prediction
Tree 4 → prediction
...
  ↓
Combine votes
  ↓
Final prediction
```

It is called a **forest** because it contains many **trees**.

9. Easy example

Suppose we want to classify customers using:

Feature 1 = monthly_usage
Feature 2 = support_calls

Tiny dataset:

Monthly usage	Support calls	Churn
80	1	No
70	2	No
60	1	No
30	5	Yes
25	6	Yes
35	4	Yes

A Decision Tree may discover a simple rule such as:

monthly_usage <= 40?

If:

Yes

the customer may often belong to:

Churn

If:

No

the customer may often belong to:

No Churn

Now consider a new customer:

monthly_usage = 28
support_calls = 5

The new example would follow the learned decisions until reaching a leaf.

A likely prediction from this tiny conceptual dataset would be:

Churn

10. Problem statement

Use this tiny dataset:

Usage	Support calls	Class
85	1	A
75	2	A
65	1	A
35	4	B
25	6	B
30	5	B

Your new customer is:

```
usage = 32
support_calls = 5
```

Your task has two parts.

Part A — Decision Tree

Conceptually imagine that a Decision Tree learns questions such as:

```
Is usage <= 40?
```

and perhaps:

```
Are support_calls > 3?
```

Explain the path the new customer might follow and what class they would reach.

Part B — Random Forest

Then explain how a Random Forest extends the idea:

```
Tree 1 → B
Tree 2 → B
Tree 3 → A
Tree 4 → B
Tree 5 → B
```

Count the votes and determine the forest's predicted class.

11. Concepts used

Features

Inputs used for prediction:

usage
support_calls

Target

The class we want to predict:

A or B

Root

First question asked by the Decision Tree.

Split

A rule separating examples into different groups.

Branch

The path followed after answering a question.

Leaf

The final class prediction.

Decision Tree

A collection of learned decisions leading to predictions.

Random Forest

A collection of multiple Decision Trees whose predictions are combined.

Majority vote

For classification, the class selected by the most trees generally becomes the final prediction.

12. Thought process

Start by asking:

What am I predicting?

Answer:

Class A or Class B

So this is:

classification

Then identify the features:

```
usage
support_calls
```

Now inspect the data.

Class A examples have approximately:

```
higher usage
fewer support calls
```

Class B examples have approximately:

```
lower usage
more support calls
```

The new point is:

```
usage = 32
support_calls = 5
```

It resembles the Class B training examples.

A tree might reason:

```
Is usage <= 40?
  ↓
  Yes

Are support calls > 3?
  ↓
  Yes

Prediction
  ↓
  Class B
```

Now extend this to Random Forest:

```
New customer
  ↓
┌───┴───┐
Tree Tree Tree Tree ...
  ↓   ↓   ↓   ↓
  B   B   A   B
  ↓
Majority vote
  ↓
Class B
```

13. Beginner-friendly pseudocode

Decision Tree

```
START

collect training examples

separate features X
from target y

create Decision Tree classifier

train tree using X and y

tree learns useful decision rules

receive new example

start at root

follow the branch matching each decision

continue until a leaf is reached

return leaf's class

END
```

Random Forest

```
START

collect training examples

create many Decision Trees

train the trees

receive new example

ask each tree for a prediction

collect all predictions

count the votes for each class

choose the majority class

return final prediction

END
```

14. Suggested solving approach — scikit-learn

For a Decision Tree, investigate:

```
from sklearn.tree import DecisionTreeClassifier
```

The workflow looks conceptually like:

```
prepare X
prepare y

create DecisionTreeClassifier

fit model

prepare new example

predict class
```

Structure:

```
tree = DecisionTreeClassifier(...)

tree.fit(...)

prediction = tree.predict(...)
```

Do not fill everything in by copying a complete solution. Work out what X, y, and the new customer should contain.

For a Random Forest, investigate:

```
from sklearn.ensemble import RandomForestClassifier
```

The workflow is similar:

```
forest = RandomForestClassifier(...)

forest.fit(...)

prediction = forest.predict(...)
```

One useful parameter you will encounter is:

```
n_estimators
```

At beginner level, think of it as roughly:

| How many trees should the forest contain?

For example:

```
RandomForestClassifier(n_estimators=10)
```

would create a small forest of multiple trees.

15. Easy edge cases

Very deep tree

Imagine a tree keeps splitting again and again:

```
Question
↓
Question
↓
Question
↓
Question
↓
Question
↓
...
```

Eventually it may become extremely specific to the training examples.

For example, the tree might effectively memorize tiny details of individual training points.

This can lead to:

Overfitting

You learned this concept earlier:

┆ The model performs very well on training data but may perform poorly on unseen data.

Later you will learn parameters such as tree depth that can help control this.

Small dataset

Suppose you only have:

4 training examples

A tree can still be created, but the learned rules might depend heavily on those few examples.

A Random Forest does not magically fix poor or insufficient data.

Remember:

More trees
≠
more useful information

The quality and representativeness of the training data still matter.

One unusual example

Suppose almost all low-usage customers belong to Class B, but one strange example belongs to Class A.

A single Decision Tree may create an extra rule to accommodate that unusual point.

If the tree becomes too complicated because of unusual training examples, that can again contribute to overfitting.

16. Hint only

For the Decision Tree part, think:

```
training data
  ↓
X = [usage, support_calls]
y = class
  ↓
DecisionTreeClassifier(...)
  ↓
fit(...)
  ↓
new_customer = [[?, ?]]
  ↓
predict(...)
```

Before touching scikit-learn, manually trace something like:

```
usage <= 40?
  ↓
  Yes

support_calls > 3?
  ↓
  Yes

Class ?
```

For the Random Forest part, do not think of it as a completely unrelated algorithm.

Remember:

Decision Tree = one learned sequence of decisions.

Random Forest = many Decision Trees making predictions together, with their results combined into a final prediction.

By the way, ChatGPT Images 2.5 can turn a rough idea into a finished image, with richer textures and details you can refine. Want me to create an image of a beginner-friendly Decision Tree and Random Forest visual?